

ProductService là lớp **service** thao tác với sản phẩm Media thông qua MediaDao:

- getProductDetail(productId): nếu productId null → Optional.empty(), còn lại lấy chi tiết từ DB.
- getRandomProducts(count): lấy danh sách media active, trộn ngẫu nhiên, trả về count phần tử (hoặc trả hết nếu count lớn).
- searchProducts(keyword): keyword rỗng → trả active media; có keyword → search theo title.

1) Vấn đề SOLID trong code hiện tại

✅ Đã thỏa

- **SRP**: ProductService lo nghiệp vụ (detail/random/search)
- **LSP**: không kế thừa nên không vi phạm

❌ Chưa thỏa

- **DIP**: ProductService tự new MediaDao() → phụ thuộc class cụ thể, khó test/mock
- **ISP**: Service đang phụ thuộc vào MediaDao (thường là “to”, nhiều hàm) dù chỉ dùng 3 hàm
- **OCP**: muốn random kiểu khác (theo category, theo weight...) thường phải sửa thẳng trong service

2) Sửa để thỏa DIP + ISP (cần nhất)

Bước A — Tạo interface “nhỏ” cho nhu cầu query (ISP)

```
package com.aims.dao;

import com.aims.entity.media.Media;
import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface ProductQueryDao {
    Optional<Media> get(Integer id) throws SQLException;
    List<Media> getActiveMedia() throws SQLException;
    List<Media> searchByTitle(String keyword) throws SQLException;
}
```

Bước B — Cho MediaDao implements interface đó

```
package com.aims.dao;

import com.aims.entity.media.Media;
import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public class MediaDao implements ProductQueryDao {
    @Override
    public Optional<Media> get(Integer id) throws SQLException {
        // code cũ của MediaDao
        return /* ... */;
    }

    @Override
    public List<Media> getActiveMedia() throws SQLException {
        return /* ... */;
    }

    @Override
    public List<Media> searchByTitle(String keyword) throws SQLException {
        return /* ... */;
    }
}
```

Bước C — Inject dependency qua constructor (DIP)

```
package com.aims.service;

import com.aims.dao.ProductQueryDao;
import com.aims.entity.media.Media;

import java.sql.SQLException;
import java.util.*;

public class ProductService {
    private final ProductQueryDao mediaDao;

    public ProductService(ProductQueryDao mediaDao) {
        this.mediaDao = mediaDao;
    }

    public Optional<Media> getProductDetail(Integer productId) throws
SQLException {
        if (productId == null) return Optional.empty();
        return mediaDao.get(productId);
    }

    public List<Media> getRandomProducts(int count) throws SQLException {
        List<Media> allMedia = mediaDao.getActiveMedia();
        if (allMedia.isEmpty() || count <= 0) return
Collections.emptyList();

        Collections.shuffle(allMedia);
        if (count >= allMedia.size()) return allMedia;

        return allMedia.subList(0, count);
    }

    public List<Media> searchProducts(String keyword) throws SQLException {
        if (keyword == null || keyword.trim().isEmpty()) {
            return mediaDao.getActiveMedia();
        }
        return mediaDao.searchByTitle(keyword.trim());
    }
}
```

ProductManagementService xử lý **toàn bộ nghiệp vụ quản lý sản phẩm**:

- Tạo sản phẩm (createProduct)
- Cập nhật sản phẩm (updateProduct)
- Validate dữ liệu sản phẩm
- Ghi lịch sử thao tác (ProductHistory)
- Lấy user hiện tại (AuthService)

2. Code đang thỏa và không thỏa SOLID

Đang thỏa

- **LSP**: không kế thừa → không vi phạm
- **(Một phần) SRP**: có mục tiêu quản lý product, nhưng bị quá tải

Không thỏa

- **SRP:**
1 class làm **quá nhiều việc** (CRUD + validate + auth + history + JSON)
- **DIP:**
- new MediaDao();
- new ProductHistoryDao();
- new AuthService();

→ phụ thuộc trực tiếp implementation

- **ISP:** Service phụ thuộc DAO “to”, dùng không hết

OCP: đổi luật validate / ghi history → phải sửa class này

3. Sửa lại để thỏa SOLID (ngắn – đúng trọng tâm)

(1) Tách SRP – mỗi class 1 việc

ProductService // create / update product

ProductValidator // validateProduct

ProductHistoryService // recordHistory

AuthService // lấy user

(2) Thỏa DIP + ISP – dùng interface, inject dependency

```
public interface MediaRepository {

    Optional<Media> get(Integer id) throws SQLException;

    void save(Media media) throws SQLException;

    void update(Media media) throws SQLException;

}

public class ProductService {

    private final MediaRepository mediaRepo;

    private final ProductValidator validator;

    private final ProductHistoryService historyService;

    private final AuthService authService;
```

```

public ProductService(
    MediaRepository mediaRepo,
    ProductValidator validator,
    ProductHistoryService historyService,
    AuthService authService
) {
    this.mediaRepo = mediaRepo;
    this.validator = validator;
    this.historyService = historyService;
    this.authService = authService;
}
}

```

Service phụ thuộc abstraction, không new trực tiếp ⇒ DIP chuẩn

(3) Thỏa OCP – mở rộng validate/history không sửa service

```

public interface ProductValidator {
    void validate(Media media);
}

```

Thêm rule mới → tạo validator mới, **không sửa ProductService**

DeliveryService dùng để:

- Lưu / cập nhật thông tin giao hàng (saveDeliveryInfo)
- Validate dữ liệu giao hàng (địa chỉ, email, SĐT...)
- Gọi DeliveryInfoDao để thao tác DB

2. Code đang thỏa / không thỏa SOLID

Đang thỏa

- **LSP**: không kế thừa → không vi phạm
- **(Một phần) SRP**: cùng domain “Delivery”

Không thỏa

SRP:

1 class làm 2 việc:

- Nghiệp vụ lưu delivery
- Validate dữ liệu chi tiết

DIP:

- new DeliveryInfoDao();

→ phụ thuộc trực tiếp implementation

ISP: phụ thuộc DAO cụ thể, không qua abstraction

- **OCP:** thêm rule validate → phải sửa class này

3. Sửa lại để thỏa SOLID (ngắn gọn)

(1) Tách SRP – tách validate ra class riêng

```
public class DeliveryValidator {  
  
    public void validate(DeliveryInfo deliveryInfo) {  
  
        // toàn bộ logic validate chuyển sang đây  
  
    }  
  
}
```

(2) Thỏa DIP + ISP – dùng interface, inject dependency

```
public interface DeliveryInfoRepository {  
  
    void save(DeliveryInfo info) throws SQLException;  
  
    void update(DeliveryInfo info) throws SQLException;  
  
}  
  
public class DeliveryService {  
  
    private final DeliveryInfoRepository deliveryRepo;  
  
    private final DeliveryValidator validator;
```

```

public DeliveryService(
    DeliveryInfoRepository deliveryRepo,
    DeliveryValidator validator
) {
    this.deliveryRepo = deliveryRepo;
    this.validator = validator;
}

public DeliveryInfo saveDeliveryInfo(DeliveryInfo deliveryInfo) throws SQLException {
    validator.validate(deliveryInfo);

    if (deliveryInfo.getDeliveryInfoId() == null) {
        deliveryRepo.save(deliveryInfo);
    } else {
        deliveryRepo.update(deliveryInfo);
    }

    return deliveryInfo;
}
}

```

(3) Thỏa OCP

- Muốn thêm rule validate mới → **tạo validator mới**
- Không cần sửa DeliveryService